



딥러닝

2. 순방향 신경망

담당교수 세종대학교 인공지능데이터사이언스학과 김정현



세종대학교
SEJONG UNIVERSITY

Contents

2

순방향 신경망

2-1 인공신경망과 딥러닝

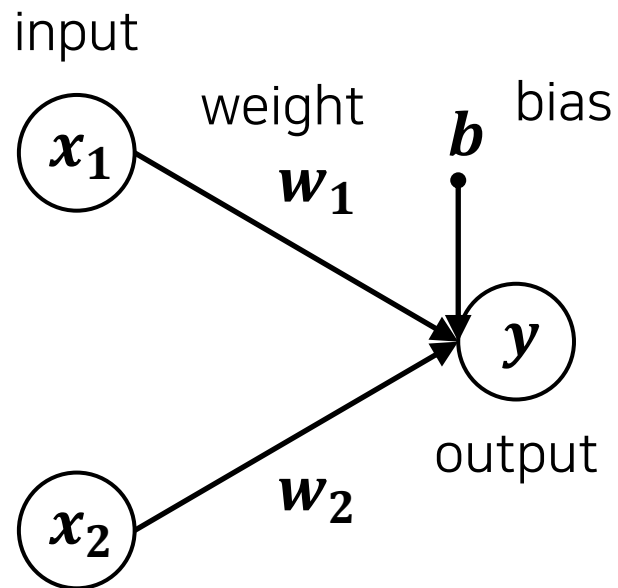
2-2 딥러닝 용어



2.1 인공신경망과 딥러닝

■ 퍼셉트론(perceptron)

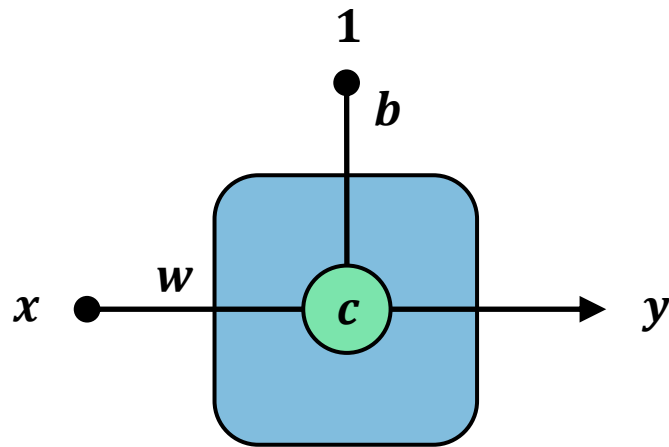
- ▶ 오늘날 인공 신경망에서 이용하는 기본 구조(입력층, 출력층, 가중치로 구성된 구조)는 프랭크 로젠블라트(Frank Rosenblatt)가 1957년에 고안한 선형 분류기



$$y = \begin{cases} 0, & w_1x_1 + w_2x_2 + b \leq 0 \\ 1, & w_1x_1 + w_2x_2 + b > 0 \end{cases}$$

2.1 인공신경망과 딥러닝

■ 인공신경망

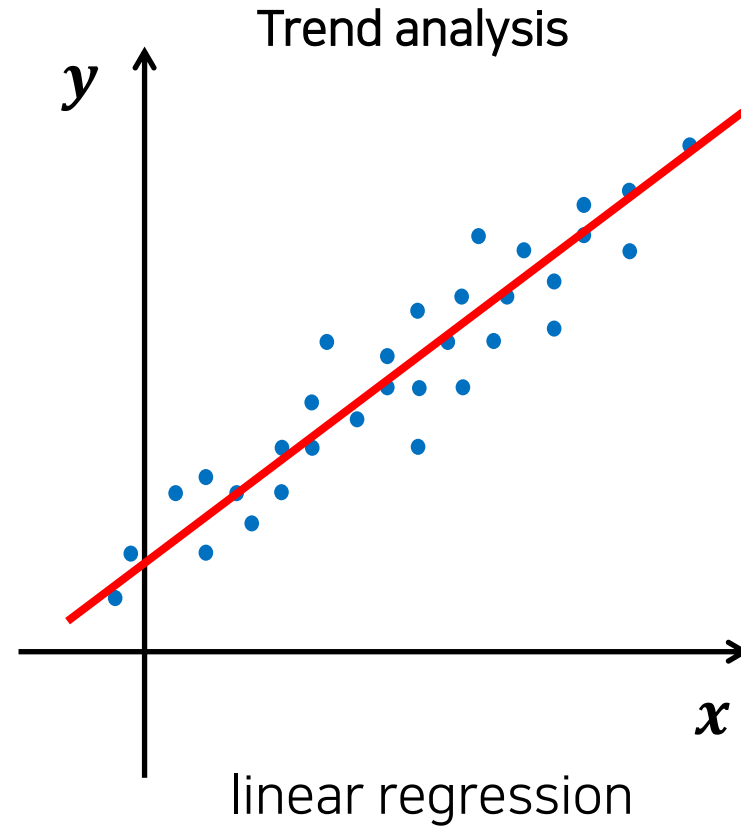
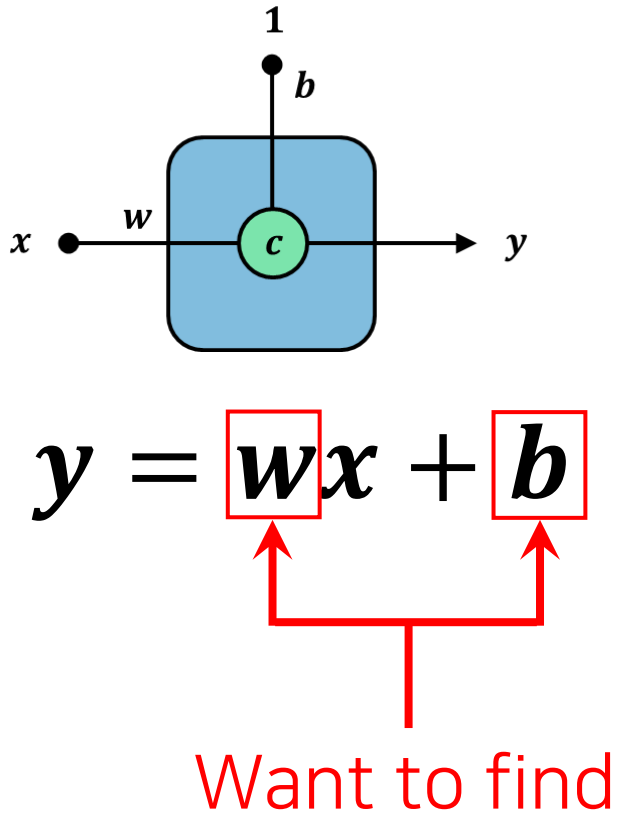


Given

$$\boxed{y} = w \boxed{x} + b$$

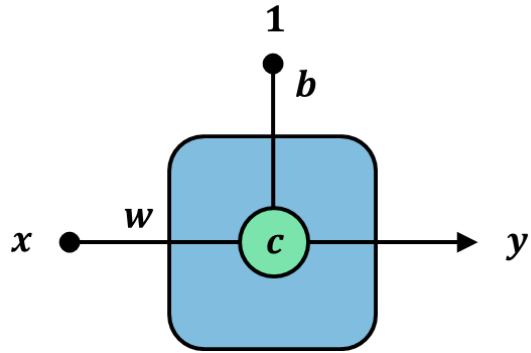
2.1 인공지능망과 딥러닝

회귀 문제를 위한 인공지능망



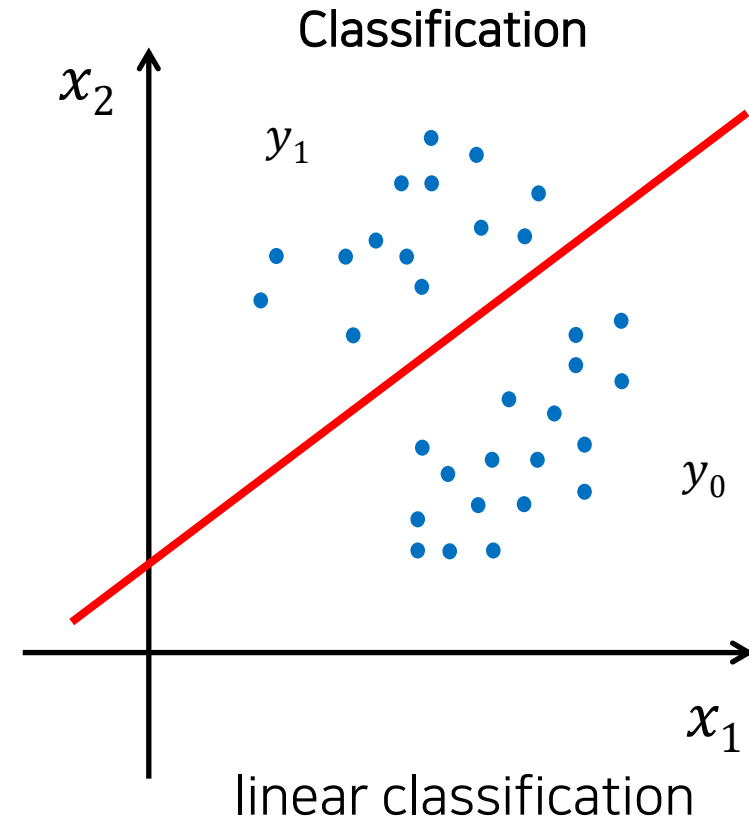
2.1 인공신경망과 딥러닝

■ 분류 문제를 위한 인공신경망



$$y = \begin{cases} 0, & wx + b \leq 0 \\ 1, & wx + b > 0 \end{cases}$$

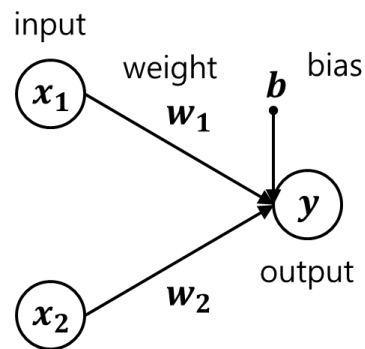
Want to find



2.1 인공신경망과 딥러닝

■ 분류 문제를 위한 인공신경망

➤ 논리 게이트 문제



AND

x_1	x_2	y
0	0	0
1	0	0
0	1	0
1	1	1

$$w_1 = w_2 = 1$$

$$b = -1$$

$$y = \begin{cases} 0, & x_1 + x_2 - 1 \leq 0 \\ 1, & x_1 + x_2 - 1 > 0 \end{cases}$$

OR

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	1

$$w_1 = w_2 = 1$$

$$b = -0.5$$

$$y = \begin{cases} 0, & x_1 + x_2 - 0.5 \leq 0 \\ 1, & x_1 + x_2 - 0.5 > 0 \end{cases}$$

NAND

x_1	x_2	y
0	0	1
1	0	1
0	1	1
1	1	0

$$w_1 = w_2 = -0.5$$

$$b = 1$$

$$y = \begin{cases} 0, & -0.5x_1 - 0.5x_2 + 1 \leq 0 \\ 1, & -0.5x_1 - 0.5x_2 + 1 > 0 \end{cases}$$

XOR

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0

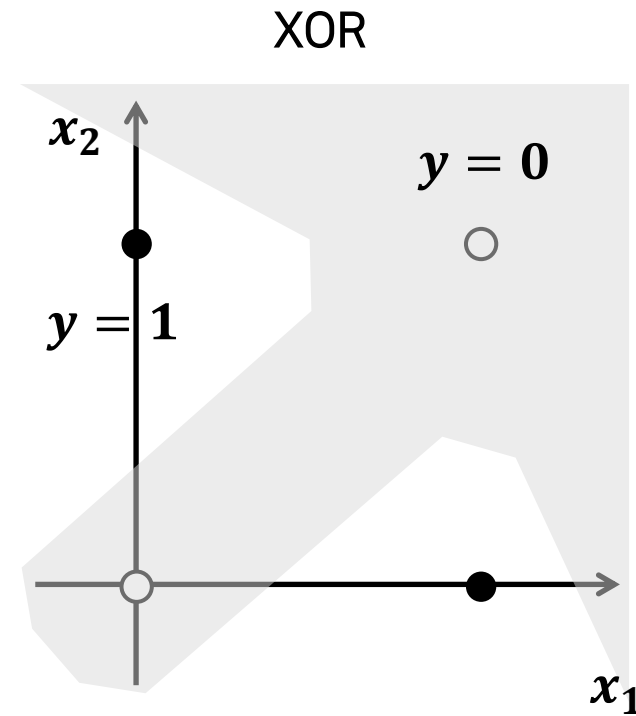
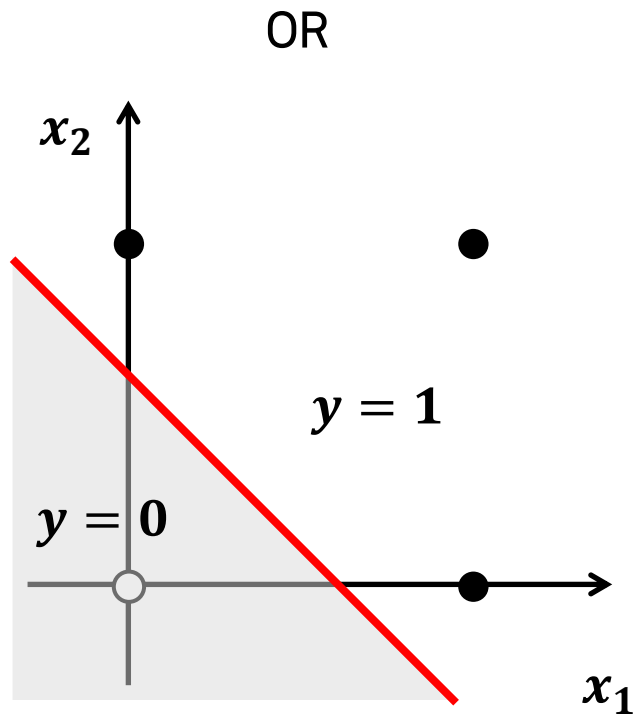
~~$$w_1 = 2, w_2 = ?$$~~
~~$$b = ?$$~~

$$y = \begin{cases} 0, & w_1x_1 + w_2x_2 + b \leq 0 \\ 1, & w_1x_1 + w_2x_2 + b > 0 \end{cases}$$

2.1 인공신경망과 딥러닝

■ 분류 문제를 위한 인공신경망

➤ 논리 게이트 문제



2.1 인공지능망과 딥러닝

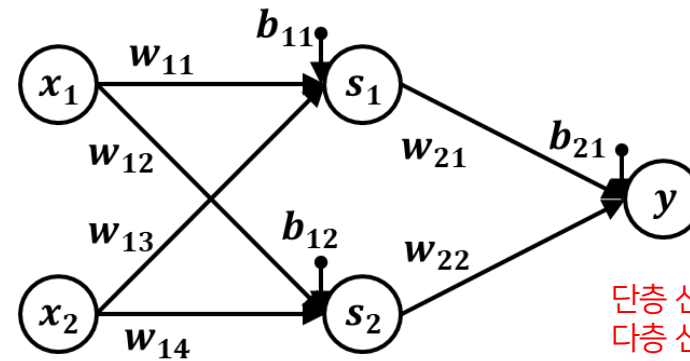
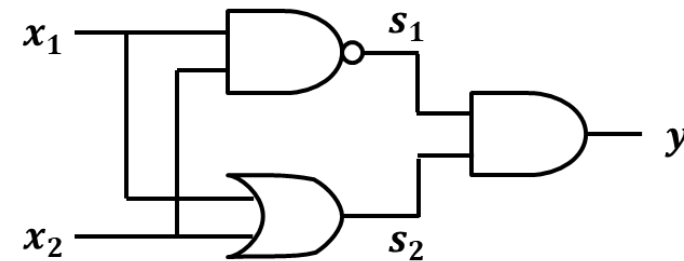
■ 분류 문제를 위한 인공지능망

➤ 논리 게이트 문제

XOR		
x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0

x_1	x_2	s_1	s_2	y
0	0	1	0	0
1	0	1	1	1
0	1	1	1	1
1	1	0	1	0

NAND: $x_1, x_2 \rightarrow s_1$
 AND: $x_1, x_2 \rightarrow s_2$
 OR: $s_1, s_2 \rightarrow y$

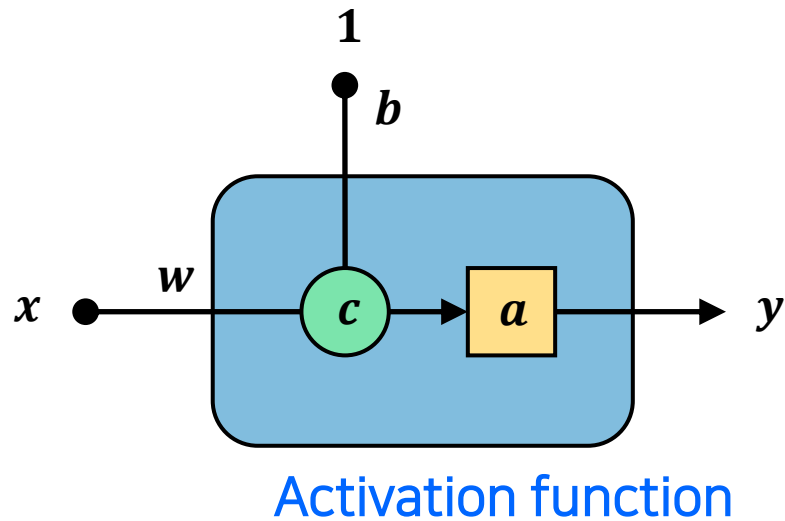


단층 신경망으로 해결 불가능한 문제를
다층 신경망으로 해결 가능

2.1 인공신경망과 딥러닝

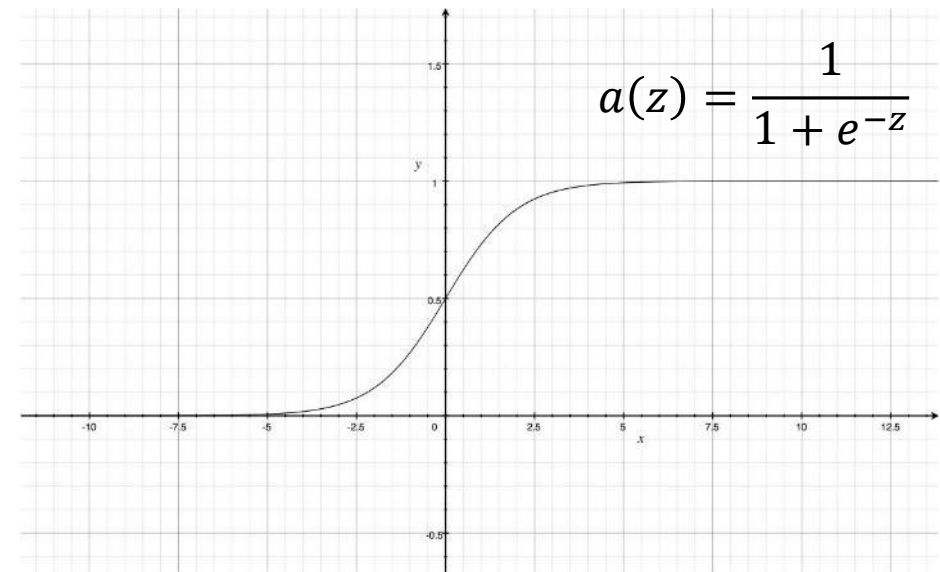
■ 인공신경망 기초

➤ 활성화함수를 통해 비선형 문제 해결



$$y = \begin{cases} 0, & a(wx + b) \leq 0.5 \\ 1, & a(wx + b) > 0.5 \end{cases}$$

A popular activation function:
Sigmoid



2.1 인공지능경망과 딥러닝

■ 인공지능경망 기초

➤ Sigmoid 활성화함수

- 이진 분류를 위한 로지스틱 회귀

- 오즈(odds)

- 어떤 사건 X 가 일어날 확률과 일어나지 않을 확률의 비율: $\frac{P(X)}{1 - P(X)}$

- 로짓(logit)

- 오즈에 로그 함수를 취한 값: $\text{logit}(P(X) = p) = \log\left(\frac{P(X)}{1 - P(X)}\right)$

- 이진 분류를 위한 로지스틱 회귀

- 로지스틱 모델에서는 가중치가 적용된 입력과 로짓 사이에 선형 관계가 있다고 가정

$$\text{logit}(p) = w_1x_1 + \dots + w_mx_m + b = \sum_{i=1}^m w_ix_i + b = \mathbf{w}^T \mathbf{x} + b$$

- 확률 p 에 관한 식으로 로짓을 변형하면

$$\log\left(\frac{p}{1-p}\right) = \mathbf{w}^T \mathbf{x} + b = z$$

$$\frac{p}{1-p} = e^z$$

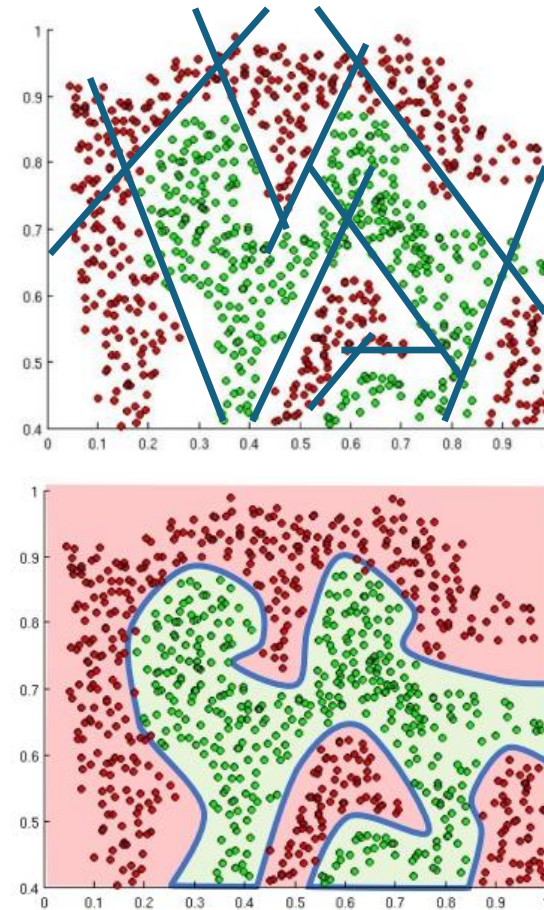
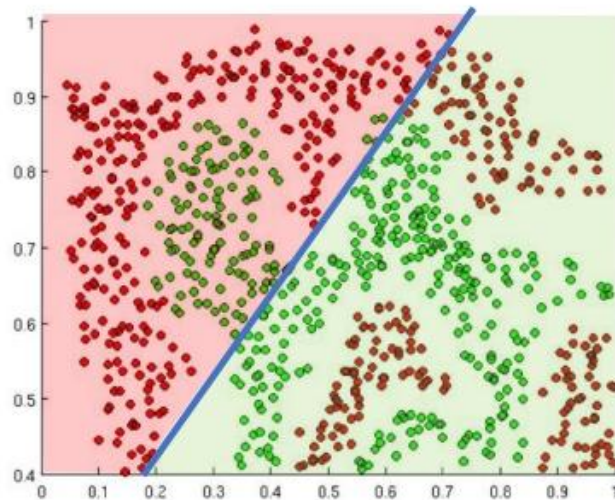
$$p = \frac{1}{1 + e^{-z}}$$

Sigmoid

2.1 인공신경망과 딥러닝

■ 복잡하고 비선형의 문제 해결 방법

- "DEEP" neural network
- Activation function



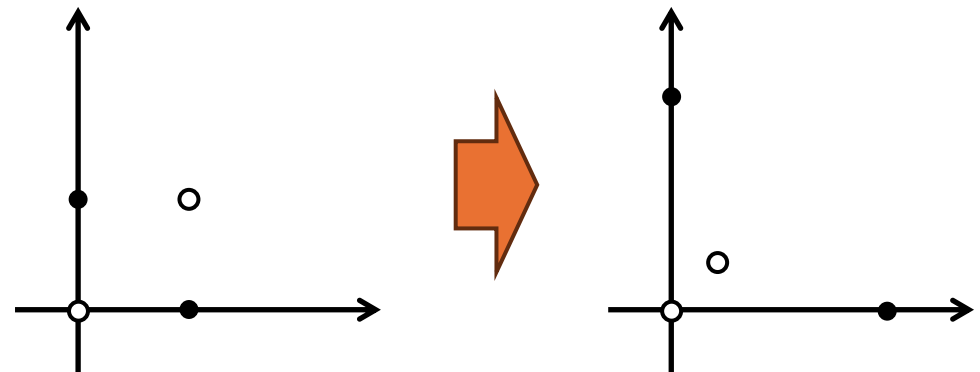
2.1 인공신경망과 딥러닝

■ 복잡하고 비선형의 문제 해결 방법(추가)

➤ 특징 공간 변환(전처리)

원래 특징 벡터	변환된 특징 벡터
$x = (x_1, x_2)^T$	$x' = \left(\frac{x_1}{2x_1x_2 + 0.5}, \frac{x_2}{2x_1x_2 + 0.5} \right)^T$

$a = (0,0)^T$	$\longrightarrow$	$a' = (0,0)^T$
$b = (1,0)^T$	$\longrightarrow$	$b' = (2,0)^T$
$c = (0,1)^T$	$\longrightarrow$	$c' = (0,2)^T$
$d = (1,1)^T$	$\longrightarrow$	$d' = (0.4,0.4)^T$



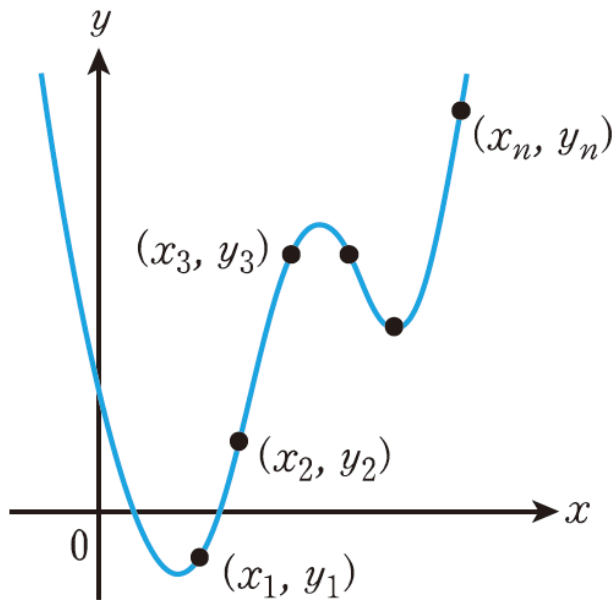
2.1 인공신경망과 딥러닝

■ 복잡하고 비선형의 문제 해결 방법(추가)

➤ 기본 심층 신경망의 변형

$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$: 데이터 샘플

$p(x) = a_0 + a_1x + a_2x^2 + \dots + a_mx^m$: 데이터의 특성을 대표하는 곡선



$$\begin{cases} a_0 + a_1x_1 + a_2x_1^2 + \dots + a_mx_1^m = y_1 \\ a_0 + a_1x_2 + a_2x_2^2 + \dots + a_mx_2^m = y_2 \\ \vdots \\ a_0 + a_1x_n + a_2x_n^2 + \dots + a_mx_n^m = y_n \end{cases}$$

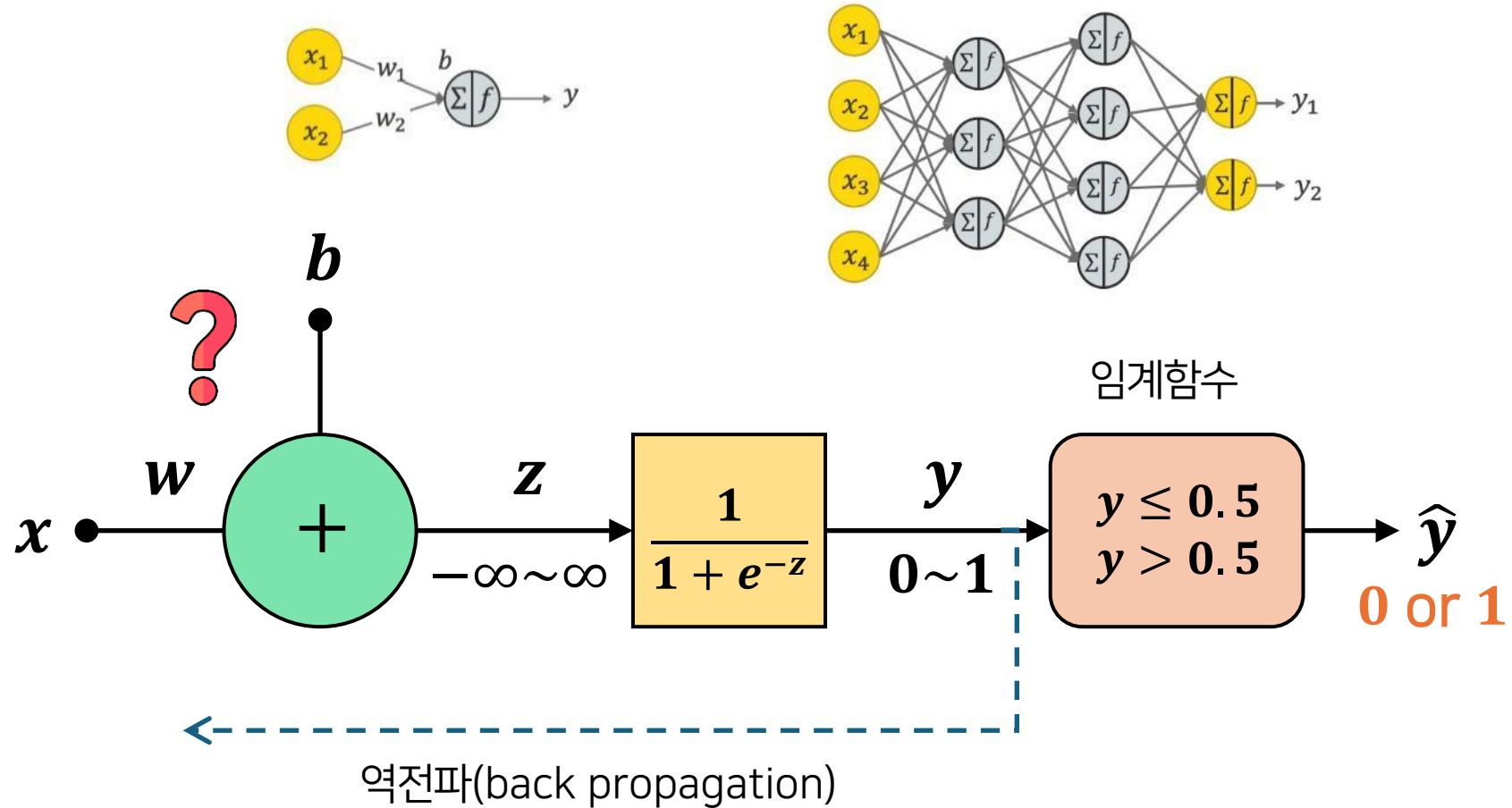
대표 곡선을 찾으려면, 샘플값 대입 후
연립선형방정식을 풀어서 계수값을 찾으면 됨

➔ **신경망 최적화로 해결**

(이를 다항 회귀라고도 함)

2.1 인공신경망과 딥러닝

인공신경망 동작 과정



2.1 인공신경망과 딥러닝

■ 인공신경망의 학습

- 역전파 과정에서 신경망 학습이 이루어짐

적합한 손실함수를 정의하고,
손실함수가 최소가 되도록 업데이트



$$\mathbf{W}^* = \arg \min_{\mathbf{W}} L(\mathbf{W}, b | \mathbf{x})$$



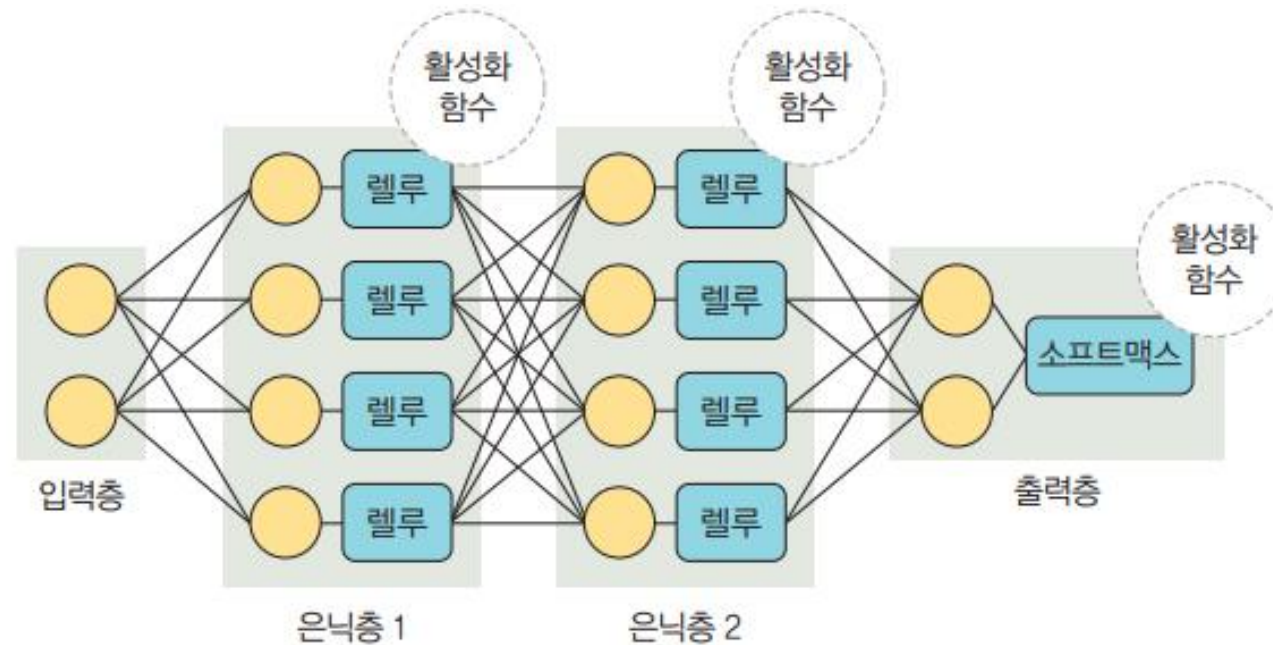
Remember:

$$\mathbf{W} = \{w_0, w_1, \dots\}$$

2.2 딥러닝 용어

■ 순방향 신경망 구조

- ▶ 다음 그림과 같이 입력층, 출력층과 두 개 이상의 은닉층으로 구성
- ▶ 입력 신호를 효과적으로 전달하기 위해 다양한 함수 사용



2.2 딥러닝 용어

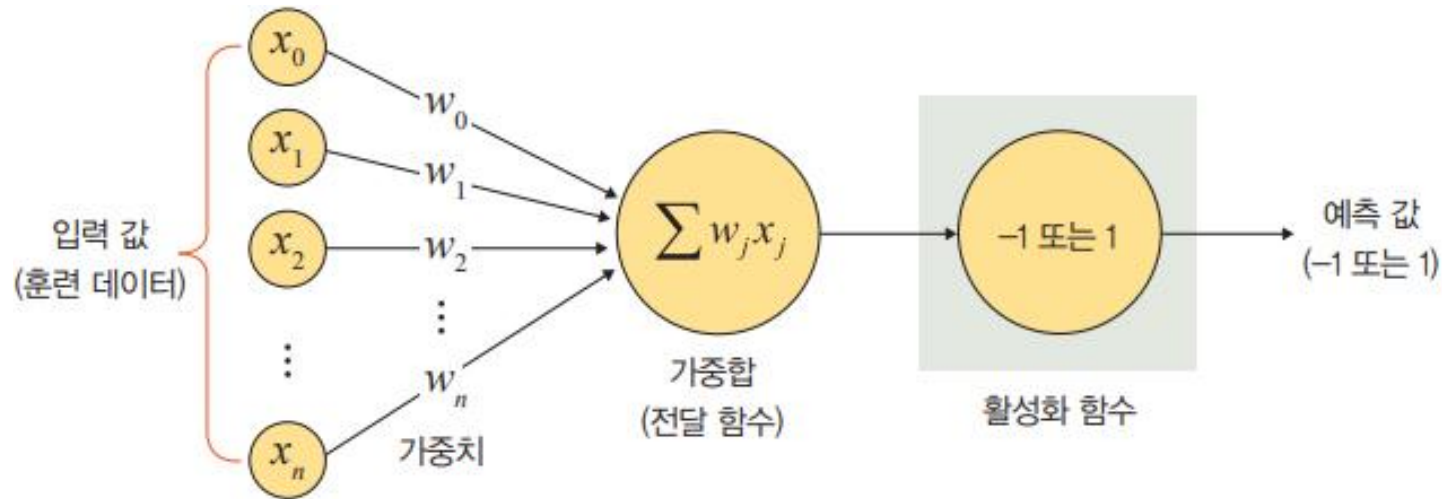
■ 순방향 신경망 구성 요소

구분	구성 요소	설명
층	입력층(input layer)	데이터를 받아들이는 층
	은닉층(hidden layer)	모든 입력 노드부터 입력 값을 받아 가중합을 계산하고, 이 값을 활성화 함수에 적용하여 출력층에 전달하는 층
	출력층(output layer)	신경망의 최종 결과값이 포함된 층
가중치(weight)		노드와 노드 간 연결 강도
바이어스(bias)		가중합에 더해 주는 상수로, 하나의 뉴런에서 활성화 함수를 거쳐 최종적으로 출력되는 값을 조절하는 역할을 함
가중합(weighted sum), 전달 함수		가중치와 신호의 곱을 합한 것
함수	활성화 함수(activation function)	신호를 입력받아 이를 적절히 처리하여 출력해 주는 함수
	손실 함수(loss function)	가중치 학습을 위해 출력 함수의 결과와 실제 값 간의 오차를 측정하는 함수

2.2 딥러닝 용어

가중치

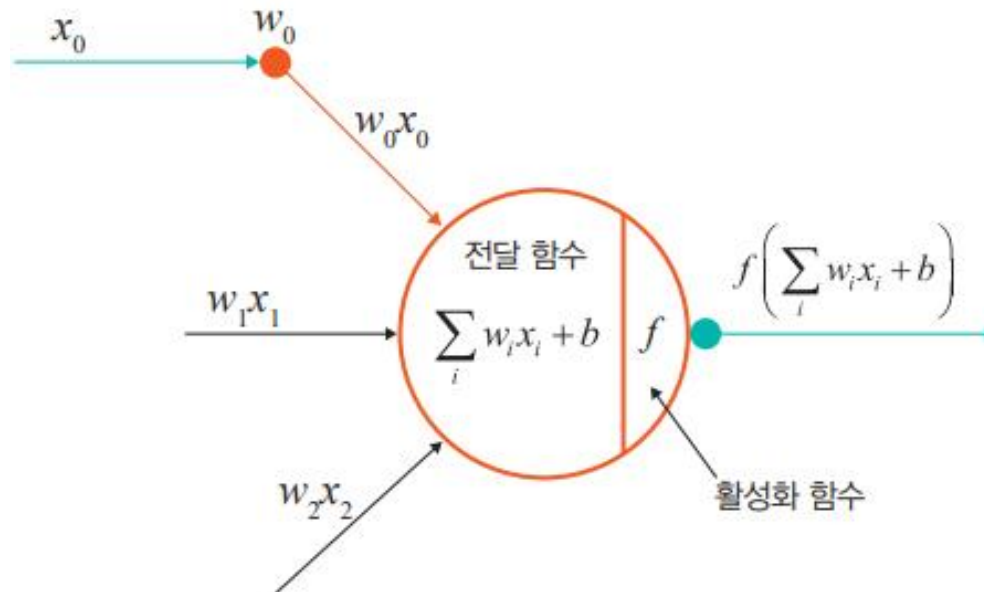
- 가중치는 입력 값이 연산 결과에 미치는 영향력을 조절하는 요소
- 다음 그림에서 w_1 값이 0 혹은 0과 가까운 값이라면, x_1 이 아무리 큰 값이라도 $x_1 \times w_1$ 값은 0이거나 0에 가까운 값이 됨
- 이와 같이 입력 값의 연산 결과를 조정하는 역할을 하는 것이 가중치



2.2 딥러닝 용어

가중합 또는 전달 함수

- ▶ 각 노드에서 들어오는 신호에 가중치를 곱해서 다음 노드로 전달되는데, 이 값들을 모두 더한 합계를 가중합이라고 함
- ▶ 노드의 가중합이 계산되면 이 가중합을 활성화 함수로 보내기 때문에 전달 함수(transfer function)라고도 함

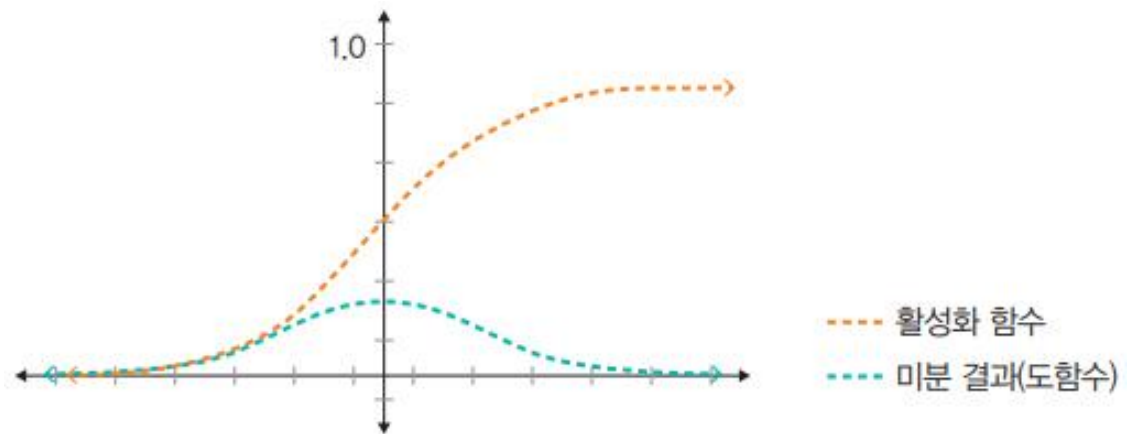


2.2 딥러닝 용어

■ 활성화 함수

- ▶ 전달 함수에서 전달받은 값을 출력할 때 일정 기준에 따라 출력 값을 변화시키는 비선형 함수
- ▶ 시그모이드(sigmoid) 함수
 - 시그모이드 함수는 선형 함수의 결과를 0~1 사이에서 비선형 형태로 변형
 - 주로 로지스틱 회귀와 같은 분류 문제를 확률적으로 표현하는 데 사용
 - 딥러닝 모델의 깊이가 깊어지면 기울기가 사라지는 '기울기 소멸 문제(vanishing gradient problem)'가 발생할 수 있음

$$f(x) = \frac{1}{1 + e^{-x}}$$

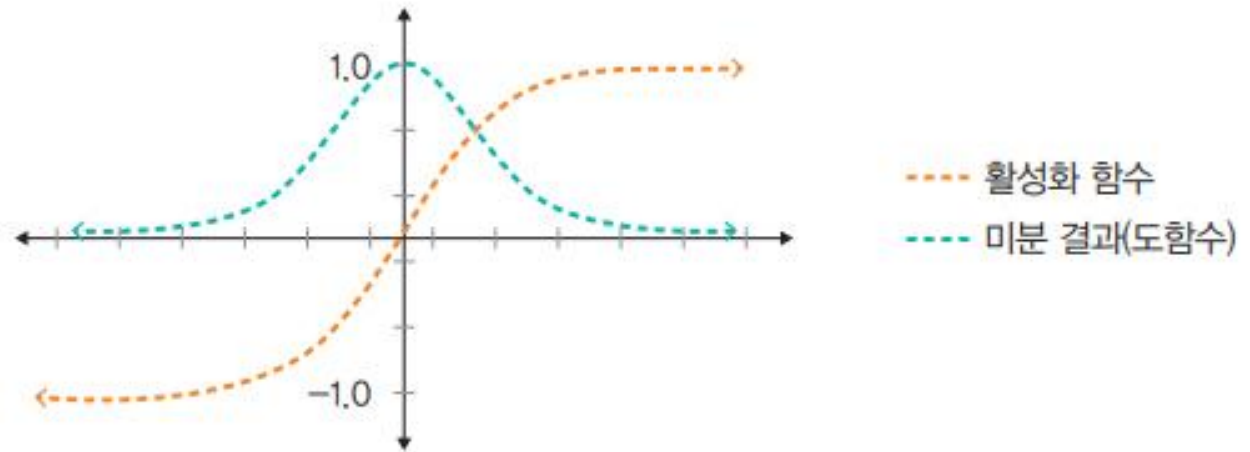


2.2 딥러닝 용어

■ 활성화 함수

- ▶ 하이퍼볼릭 탄젠트(hyperbolic tangent) 함수
 - 선형 함수의 결과를 -1~1 사이에서 비선형 형태로 변형
 - 시그모이드에서 결괏값의 평균이 0이 아닌 양수로 편향된 문제를 해결하는 데 사용했지만, 기울기 소멸 문제는 여전히 발생

$$f(x) = \tanh x = \frac{\sinh x}{\cosh x} = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



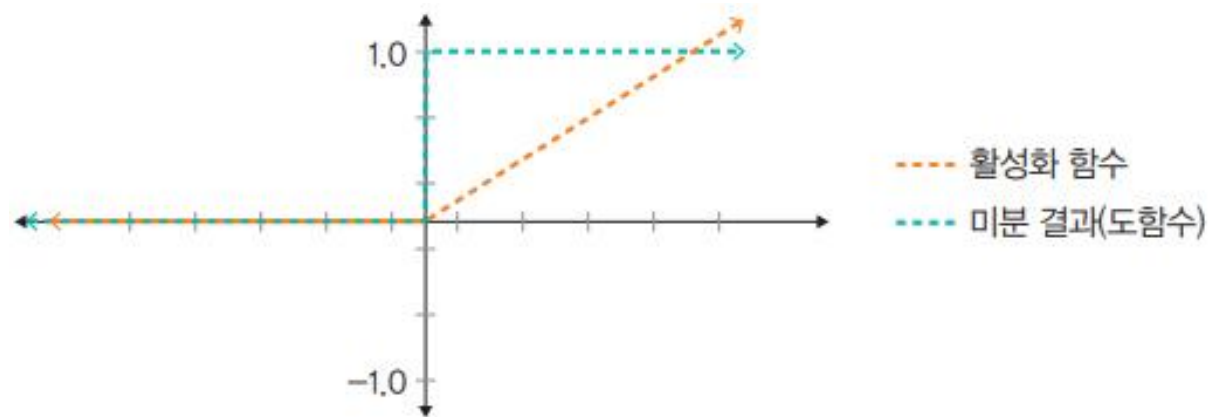
2.2 딥러닝 용어

■ 활성화 함수

> 렐루(ReLU) 함수

- 입력(x)이 음수일 때는 0을 출력하고, 양수일 때는 x 를 출력
- 경사 하강법(gradient descent)에 영향을 주지 않아 학습 속도가 빠르고, 기울기 소멸 문제가 발생하지 않는 장점이 있음
- 일반적으로 은닉층에서 사용되며, 하이퍼볼릭 탄젠트 함수 대비 학습 속도가 6배 빠름
- 문제는 음수 값을 입력받으면 항상 0을 출력하기 때문에 학습 능력이 감소하는데, 이를 해결하려고 리키 렐루 (Leaky ReLU) 함수 등을 사용

$$f(x) = \max(0, x)$$



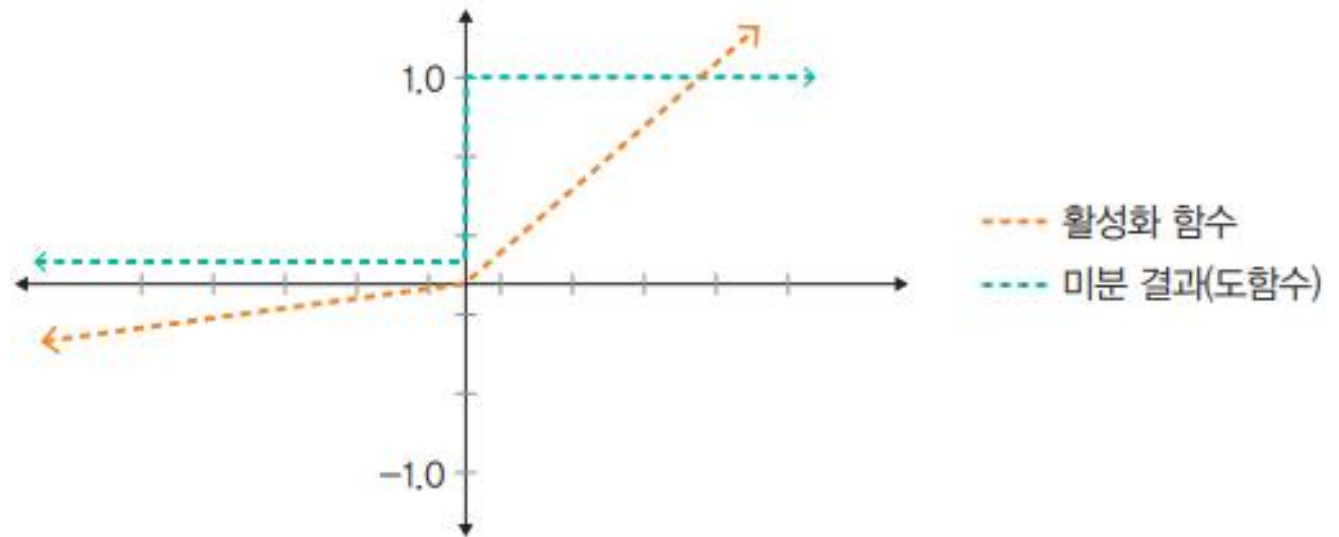
2.2 딥러닝 용어

■ 활성화 함수

> 리키 렐루(Leaky ReLU) 함수

- 입력 값이 음수이면 0이 아닌 0.001처럼 매우 작은 수를 반환
- 이렇게 하면 입력 값이 소멸되는 구간이 제거되어 렐루 함수를 사용할 때 생기는 문제를 해결할 수 있음

$$f(x) = \max(\alpha x, x)$$



2.2 딥러닝 용어

■ 활성화 함수

> 소프트맥스 함수

- 소프트맥스(softmax) 함수는 입력 값을 0~1 사이에 출력되도록 정규화하여 출력 값들의 총합이 항상 1이 되도록 함
- 소프트맥스 함수는 보통 딥러닝에서 출력 노드의 활성화 함수로 많이 사용
- 수식으로 표현하면 다음과 같음

$$y_k = \frac{\exp(a_k)}{\sum_{i=1}^n \exp(a_i)}$$

- n 은 출력층의 뉴런 개수, y_k 는 그 중 k 번째 출력을 의미
- 분자는 입력 신호 a_k 의 지수 함수, 분모는 모든 입력 신호의 지수 함수 합으로 구성

2.2 딥러닝 용어

■ 손실 함수

- ▶ 경사 하강법은 학습률(η , learning rate)과 손실 함수의 순간 기울기를 이용하여 가중치를 업데이트 하는 방법
 - 즉, 미분의 기울기를 이용하여 오차를 비교하고 최소화하는 방향으로 이동시키는 방법이라고 할 수 있음
- ▶ 이때 오차를 구하는 방법이 손실 함수
 - 즉, 손실 함수는 학습을 통해 얻은 데이터의 추정치가 실제 데이터와 얼마나 차이가 나는지 평가하는 지표라고 할 수 있음
- ▶ 이 값이 클수록 많이 틀렸다는 의미이고, 이 값이 '0'에 가까우면 완벽하게 추정할 수 있다는 의미
- ▶ 대표적인 손실 함수로는 평균 제곱 오차(Mean Squared Error, MSE)와 크로스 엔트로피 오차(Cross Entropy Error, CEE)가 있음

2.2 딥러닝 용어

■ 손실 함수

> 평균 제곱 오차를 구하는 수식

- 데이터 샘플에 대한 평균을 의미하는 손실 함수

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

$$\left(\begin{array}{l} i: \text{데이터 샘플 인덱스} \\ y_i: \text{정답 레이블} \\ \hat{y}_i: \text{신경망의 출력(신경망이 추정한 값)} \end{array} \right)$$

2.2 딥러닝 용어

■ 손실 함수

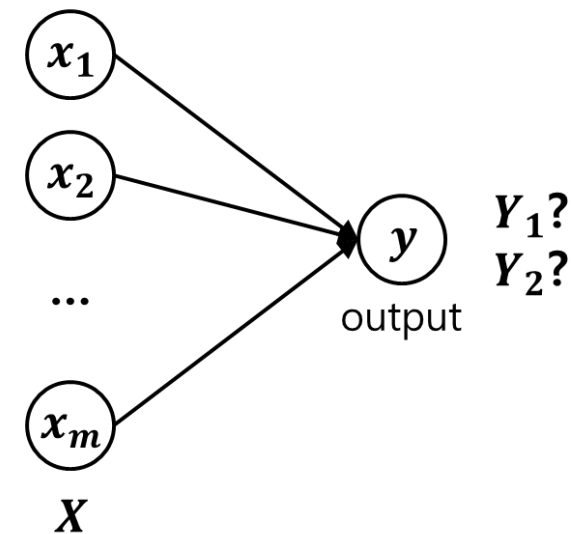
> 크로스 엔트로피 오차

- 바이너리 크로스 엔트로피를 구하는 수식
 - 데이터 샘플에 대한 평균을 의미하는 손실 함수
 - 이진 분류에 사용(클래스 구분이 1 또는 0)

$$\text{BinaryCrossEntropy} = -\frac{1}{N} \sum_{i=1}^N (y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i))$$

i : 데이터 샘플 인덱스
 y_i : 정답 레이블
 $\hat{y}_i$: 신경망의 출력(신경망이 추정한 값)

Binomial classification



2.2 딥러닝 용어

■ 손실 함수

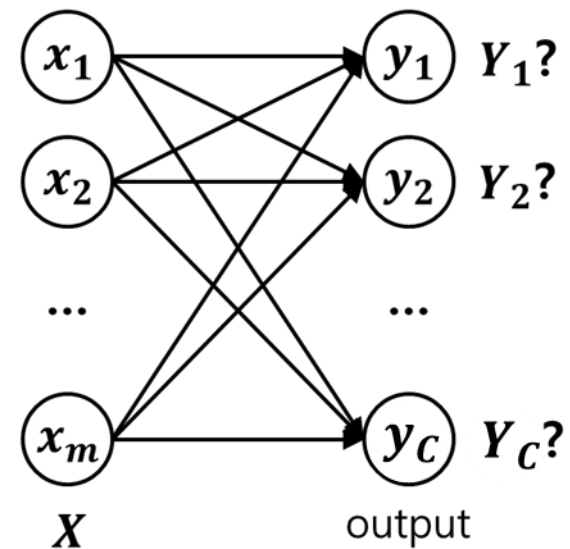
> 크로스 엔트로피 오차

- 크로스 엔트로피를 구하는 수식
 - 데이터 샘플에 대한 평균을 의미하는 손실 함수
 - 다중 분류에 사용(클래스 구분이 1의 위치)

$$CrossEntropy = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C (y_{i,j} \log \hat{y}_{i,j})$$

i : 데이터 샘플 인덱스
 j : 클래스 인덱스
 $y_{i,j}$: 정답 레이블
 $\hat{y}_{i,j}$: 신경망의 출력(신경망이 추정한 값)

Multinomial classification



2.2 딥러닝 용어

■ 손실 함수

- Mean squared error

$$L(W) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Regression

- Binary cross entropy

$$L(W) = -\frac{1}{N} \sum_{i=1}^N (y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i))$$

Classification

- Cross entropy

$$L(W) = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C (y_{i,j} \log \hat{y}_{i,j})$$

2.2 딥러닝 용어

손실 함수

Binary cross entropy 수식 유도

- 로지스틱 회귀에서 가능도(likelihood) 함수
 - 데이터셋의 각 샘플이 독립적이라고 가정
 - 이진 분류에서 레이블 1을 베르누이(Bernoulli) 변수로 생각할 수 있음

$$l(\mathbf{w}, b | \mathbf{x}) = \prod_{i=1}^N p(y^{(i)} | \mathbf{x}^{(i)}; \mathbf{w}, b) = \prod_{i=1}^N \left(\sigma(z^{(i)}) \right)^{y^{(i)}} \left(1 - \sigma(z^{(i)}) \right)^{1-y^{(i)}}$$

- 로지스틱 회귀에서 로그 가능도 함수
 - 로그 함수를 적용하면 가능도가 매우 작을 때 일어나는 수치상의 언더플로를 미연에 방지 가능
 - 곱 연산을 합 연산으로 바꿀 수 있음

$$\begin{aligned} L(\mathbf{w}, b | \mathbf{x}) &= \log l(\mathbf{w}, b | \mathbf{x}) = \sum_{i=1}^N \left(y^{(i)} \log \left(\sigma(z^{(i)}) \right) + (1 - y^{(i)}) \log \left(1 - \sigma(z^{(i)}) \right) \right) \\ &= \sum_{i=1}^N \left(y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right) \end{aligned}$$

(추가)
 샘플 갯수로 정규화(1/N 배)
 손실함수를 최소화하는 방향으로 정의($\times -1$)

경청해 주셔서 감사합니다.